SUPERDOCS · ROUND 2

THE GTM / GROWTH ENGINEER TASK DOCUMENT

This is the growth document, for the GTM / Growth Engineer role. If you applied for both roles, our engineer document travels alongside this one, and the email that brought them explains the pair.

SuperDocs, in one line: think Claude Code or Cursor, but for documents. Why that and not a chat window: getting words written is a solved problem; editing the document you already have is not. A chat window hands you text to carry back by hand. Here the work lands in the document itself, and that closing of the loop is the product.

Any level of experience is welcome here, and if you bring more than the posting asked for, we welcome that too. The task is the real test, and it always was; it replaces the call. What we read it for: creativity, agency, grit, and technical ability, and the first two weigh more than the last.

No whiteboards, no algorithm quizzes, no marketing-theory quizzes, no BS rounds. One task, with parts inside, and it is deliberately big. Do it well compared to your peers, and I will talk with you directly soon after.

One sentence frames everything in this document: the only way to judge a growth person is to watch them grow something.

GETTING STARTED

Use the product properly before you build anything around it, on real work of your own. Sign up at use.superdocs.app; there is no guest mode, and everything ahead assumes you have an account and have actually used it. The first instruction in a fresh session can be slow, or can fail once with a visible error while things warm up. If you see an error, send it again and it settles.

If you work with a coding agent, it can work with SuperDocs too: point Claude Code, Cursor, or anything that speaks MCP at superdocs.app and it can sign itself up and build documents on its own. For this document that whole side is available and never required; nothing ahead assumes anything beyond the web app you have already used. The developer docs at docs.superdocs.app are optional the same way: not expected, and quietly telling if you enjoy them.

One practical note: open sign-in links in a real browser, not inside the LinkedIn or Instagram app. In-app browsers break sign-in pages.

TASK 1. BUILD A GROWTH MACHINE

Every growth candidate in this round gets this brief, and every growth submission answers it, so this is the one place we compare all of you side by side. It is sized on purpose like work for a small team: the role you are applying for is one person, with AI, doing the work of many. Read that size as the point, not as a warning. If you hold our engineer document as well, do the shared task of the role you are really going for; doing both earns extra credit and is never required.

The job description asked how you would make our entire GTM operation run itself, with agents doing the work of 20 to 100 people while humans only steer, and it was not a rhetorical question. It keeps its own place later in this round. This task is one working piece of that machine, built by you and shown running.

THE PROBLEM: THE FIRST TEN, THEN THE MACHINE

SuperDocs has grown the way small products grow: someone finds it, uses it, and tells someone else. Nobody's full-time job has ever been to change that. Somewhere out there is a specific group of people who need this product badly and have never heard of it. Find them, and build the thing that reaches them.

First, the audience. Pick one specific group, not an industry: the kind you could name ten of and reach this week. Tell us who they are, what they do today instead of using SuperDocs, what would genuinely make them switch, and how you would find the first ten. Finding them is the task here; contacting them is not. If you name names, name companies, never private individuals; and if your audience is individual people rather than organizations, describe them by role and by where they gather, not by personal name. You already told us how you would grow SuperDocs when you applied. Start from your own plan or pick a better idea; changing your mind is fine, and explaining why counts in your favor.

Second, the machine. Build the working system that would reach that audience, in whatever channel they actually live in: an outbound machine that finds them, enriches them, and writes to each one like a human who did the reading; a content or search machine that gets found by them; a community or launch machine that earns their attention; a product loop that turns users into more users. One channel built properly beats three channels sketched. The machine must actually run, end to end, and then run again on a fresh batch without you patching it in between. A fresh batch means new inputs of the same kind: new prospects for outbound, new topics for content, new posts for community. Record both runs, because your Task 4 demo video is where we watch it work. It runs on public or synthetic targets only: made-up prospects, public information, accounts and inboxes you own. The naming rule above covers everything your machine stores or produces: real companies may appear anywhere in your repository, real people may not.

And one bright line, because growth tasks touch the real world: nothing your machine produces is sent to any real person or any real company. In our name, never; in your own name, not while this round runs. No cold emails, no DMs, no messages to scraped lists. We do all outreach ourselves, always. Sharing what you built on your own channels, meaning public posts on accounts you own, is a different thing, covered later in this document, and always your choice; even then, nothing lands in a real inbox or DM except your own. Restraint is part of what we are measuring, and a submission that contacted any real person or company fails this round, no matter how good the work is.

Third, the piece. Show one real piece of work your machine produces or carries out: a landing page, a cold email sequence, a launch post, or a campaign plan. Your call, and if your channel calls for a different shape, make the matching piece for it. This is where taste shows: would the audience you picked actually stop and read this?

Fourth, the numbers. Write one honest page on measurement: which numbers would prove this machine works, where it leaks, what breaks first if it ran at ten times the volume, and where a human must stay in the loop. If you ran a real experiment while building, on your own channels or on synthetic targets, and it

produced nothing, write it up anyway, rigorously. An honest null result with a clear log beats a good number nobody can explain.

Those four parts are the floor, and none of them is optional. Everything beyond them is extra credit, and building beyond them is welcome. If you must choose where to spend yourself, choose depth: a small machine that genuinely runs beats a grand one that exists only in slides, and telling us what you cut and why costs you nothing.

WHAT SEPARATES A STRONG MACHINE

An audience choice specific enough to be wrong, defended anyway. Personalization a stranger would mistake for hand-written care. A machine that behaves the second time exactly as it did the first. Numbers that show you know what each stage costs and loses. And claims that never outrun the product: whatever your machine produces promises nothing SuperDocs does not do today, and what the app does today, with the documentation as its spec, is the ceiling; the marketing site also paints where we are going, and is not the spec. When in doubt, say less and show more.

THE TOOLS

Use whatever reaches your audience: enrichment tools of the Clay or Apollo class, automations built with n8n, Make or Zapier, outbound or conversation bots, analytics stacks, content pipelines, plain scripts in any language, or no code at all. Nothing on that list is required, and nothing paid is expected: free tiers are enough, and we judge the build, not your tooling budget. SuperDocs has a REST API and an MCP server if your machine needs a document engine behind it; using them is welcome and never required, and this task assumes nothing beyond the web app you have already used. If a tool you genuinely need sits behind a paywall you cannot cross, write to us and we will sort something out.

WHAT WE ACTUALLY READ

Who you chose and why. Whether the machine genuinely runs, twice. The taste in what it produced. The honesty of your measurement. The restraint you kept. We are reading for judgment and instincts, not for a business plan, and concrete beats comprehensive. This brief will leave you with open questions, and that is by design: decide for yourself, note the decision and your reasoning in a notes file in your repository or in your Task 4 write-up, and keep moving. A noted assumption counts in your favor. If you hit a wall you truly cannot get around, write to us.

WHERE IT GOES

This task lives in a private GitHub repository that you create. Name it doctask-your-name, doctask-priya-sharma for example, and keep SuperDocs out of the name: this machine is yours, and when the round is over it should carry your name, not our brand. Add github.com/o-kadam (Omkar, the founder; this exact username) as a collaborator so we can see what you built, and drop the repository link into your submission too, so nothing goes missing if an invitation expires; re-send the invitation when you submit the form and we accept promptly. If you are doing the engineer shared task as well, both tasks live in this same repository, in two clearly named folders. A repository is just a folder with history, and it does not need to contain code:

who you chose and why, exported configurations, screenshots, the produced piece, and your measurement page are a complete submission for a no-code machine. If the job post led you to expect only your work and a demo video, GitHub is the one new mechanic, in two small pieces: this private repository, and a pull request for what you build on SuperDocs. Both are smaller than they sound, and neither needs an install: all of it works on github.com in the browser, new repository, drag your files in, Settings then Collaborators. A coding assistant such as Claude Code or Codex walks you through it too if you prefer, it is a normal routine, and learning it fast is itself part of what we measure.

All growth candidates hold one brief, and a public repository would hand later submissions the earlier ones to copy. Private for fairness, not secrecy. Everything in that repository remains yours: we open it to evaluate your submission, and for nothing else.

Your demo video and your write-up belong to Task 4, so nothing more is needed here. One deadline covers this whole round, every task in it, and that deadline lives in the email that brought you this document.

TASK 2. YOUR ASSIGNED BUILD

Use the product properly, on real work of your own. Then build the thing below. This build was chosen for you, matched to what your application told us about your skills and interests, and it is not optional.

YOUR ASSIGNED BUILD: Dependency map

WHO IT SERVES

Growth and engineering

WHAT TO BUILD

Find open-source projects and products that depend on document manipulation libraries. Heavy dependency usually means a homegrown pipeline that is expensive to maintain.

WHAT STRONG LOOKS LIKE

uses public package and code search properly, and separates casual use from a real pipeline.

Value: the business brief says a company that built its own editing pipeline is often a prime target. This finds them mechanically.

Output: ranked table plus the query method.

DIFFICULTY BAND

S2

SURFACES IT TOUCHES

chat, search, export

WHERE THE FINISHED WORK GOES

the submission form, into Drive

A few promises that hold for whatever your card asks. It can be completed without building a backend. SuperDocs has a REST API and an MCP server if your build wants a document engine behind it, and plenty of the assigned builds need neither; using them is welcome and never required. If partway through you find

your build needs more engineering than you expected, say so plainly in your write-up; that is information we want, not a mark against you.

The difficulty band on your card describes the build, not you. We judge within a band, so a simpler build done immaculately beats a harder one left half done.

If your assigned build genuinely does not fit you, tell us and we will work it out.

Two practical notes. First, spend your operations deliberately: try ideas on a small sample before running anything across a whole document or a whole batch, and give anything that loops a clear stopping rule. Second, big documents and deep-thinking runs can work silently for thirty seconds to a few minutes; a quiet screen with no error means it is still processing, not that it crashed. Wait it out rather than resending; retries spend operations.

What else you may build around this card, and the extra credit for it, comes next.

THE THREE KINDS OF WORK

Everything in this round is one of three kinds of work. The first kind is assigned: Task 1, the build or builds on your card, Task 3, your clip, and Task 4. Do those; they are not optional. The second kind is the shared open list, attached alongside this document: one list for every candidate in both roles, and you may pick anything on it that pulls you, growth-shaped or engineer-shaped; the lanes are not fences. Nothing on the list is claimed or reserved. If two people build the same thing, both count, and nobody is blocked by anybody. The third kind is your own invention: bring a use case we never wrote down, on one condition: it has to be a genuinely proven real-world case, the kind with actual companies you can name who might want it.

Everything beyond the assigned work is voluntary and earns extra marks; do as many extras as you want, including none. If your assigned build leaves you cold, finish it to its honest minimum and pour the rest of your energy into something from the list or something of your own, and submit both. The extras we most hope to see: integrations with the tools people already work in, of the Slack and Google Docs class; additions to coding workflows, of the Cursor and VS Code class; and ambitious original tools. Whatever you make, build ON SuperDocs, never a copy of it. A clone says nothing about your creativity; it only says you own a subscription.

TASK 3. USE CASES, AND WHO WOULD BUY THEM

Map about ten real use cases for SuperDocs, and for each one name the companies that would actually pay for it. Real companies, researched: what they do, why this use case bites them, and which role inside such a company would care. Companies and roles only, never named people, and never anyone's contact details. Then sketch how a first conversation would start: where these companies listen, and what would open the door.

This is judged on whether you can find a real buyer, not on how many industries you can list. Two protections, printed here because they matter. First: we do all outreach ourselves, always. You map; you never contact anyone, and a submission that contacted any real person or company fails this round, no matter how good the work is. Second: an honest empty answer is safe. If your digging says a market is a poor fit for us, that finding, shown with evidence, scores as well as ten tidy rows.

And if one of your mapped use cases genuinely excites you, building a small demo of it counts as an extra, with the extra marks to match.

THE CLIP TASK

One more piece is personal to you, and it is small on purpose. Make one clip, fifteen to thirty seconds long, about one SuperDocs feature: the feature named for you below.

Your feature: File to finished file, timed

The clip is a screen recording of that feature actually working, with your own voice over it. In that half minute, explain it, demonstrate it, and sell it. Showing your face is optional; presence and delivery done well earn extra points, and a clean voice-over is completely fine.

Keep numbers and counts out of it. Show what the feature does; do not quantify it.

This clip is separate from your demo video in Task 4, and it is not for posting anywhere. It reaches us only through the submission form: upload the file, and add an unlisted link as well if you have one. The form includes a short consent about how we may use it; nothing of yours is used without that consent. Do not publish it on your own channels; sharing your OWN build publicly is a different thing, covered later in this document.

Half a minute to make one feature land is the growth job in miniature: choosing what to show, what to say, and what to leave out. Treat it as seriously as its size is small.

TASK 4. THE DEMO VIDEO AND THE WRITE-UP

Going to market is most of this role, so this is where we see whether you can explain your own work to someone who has never seen it.

Record one demo video covering what you built this round: your machine from Task 1 with its two runs, your assigned build or builds, and any extras worth showing. The clip task stays separate. Aim for about three minutes; five at the very most. Being on camera is optional and encouraged: presence, presentation and delivery done well earn extra points, and a clean voice-over alone is completely fine. Before you record, clear the frame: no credentials, no terminal output, no private documents on screen; if something must be hidden, cover it with a solid opaque box, because blurs and pixelation can be reversed. Submit the video through the form: upload the file, and add an unlisted link as well if you have one. If we ever feature your work, we host our own copy, always with credit to you, and always after asking you first.

Alongside the video, write one page about what you built: what it is, the calls you made, what you cut and why, and your honest numbers. Real numbers, however small, beat vague claims; and if you have none, say what you would measure instead of inventing any. One page a stranger understands is the test.

EXTRA CREDIT

The bottom of this round is open. Build something extra, record something extra, do something cool we never asked for; the submission form has a field just for it, and it earns extra marks. That same field is the

home for anything you already built on SuperDocs before this document reached you: tell us, link it, it counts. The one thing that earns nothing, here or anywhere, is a clone of SuperDocs.

THE QUESTIONS

The submission form asks every candidate the same short set of questions. They are printed here so you can think while you build; answers go in writing on the form when it arrives, and the strongest get talked through with me live.

One. What broke. Tell us every bug, rough edge, and confusing moment you hit while working on this round. Be blunt; this is among the most useful things you can send us, and we read all of it. Bugs you find and report count in your favor: report them through the bug button in the app's side rail when you hit them, or by replying to the email that brought you this document.

Two. If you were running this company, what one number would you watch every morning, and why that one?

Three. The five features you would build next, in order, and the one thing you would drop; and what frictions or bugs would you fix immediately.

Four. How would you make our entire GTM operation run itself, a system where agents do the work of 20 to 100 people and humans only steer? Be concrete: what loops, what tools, what checks, where humans stay in the loop, and what breaks first. You met this question when you applied; this is where it lands.

Five. Look at superdocs.app and docs.superdocs.app with a growth person's eyes, the way you were told you would be asked to: what would you change? No developer background needed; this is a fresh-eyes read of our public surfaces, not a docs test. Honest and specific beats polite, and criticism scores in your favor here.

And one open invitation, never required and not graded: we are collecting thoughts on our name and brand. If naming is your kind of thing, tell us what you would call this company: a name that sets us apart, with an eye on which domains are actually available, and whatever branding advice comes with it. Run your thinking as wide as you like. If we ever use a name you suggested, an Amazon gift card comes your way as a thank-you. We are not announcing a rename; we are borrowing your taste.

HOW TO WORK ON THIS

This task is intentionally long and intentionally huge, and the reason is honest: a very large number of people applied, and I want to spend my review time on serious, high-caliber people. We are looking for people who can do the work of many, using AI heavily. You do not need a full week of free time; a couple of weekends on your own terms will do it. Few will finish. Finishing puts you among the few; do it well compared to your peers, and I will talk with you directly soon after.

We expect you to use AI heavily. Use every AI tool you have; that is the point. Build with whatever you have; we judge the build, not your tooling budget. Using AI well is the job, not a shortcut around it.

A few working methods we use ourselves, offered, not mandated. Keep a TASK.md that says how to work with you and a PROGRESS.md where you log the assumptions you make as you go. Make your long runs

resumable: checkpoint your work so a crash or a context reset never loses it, and save immediately, always. When you verify your own work, let a fresh pair of eyes do it: a verifier that is not the implementer catches what the author cannot. And when you write up your work, document the trade-offs and limitations you know about instead of burying them; a named limitation reads as judgement, a hidden one reads as a miss.

One promise from the job description holds everywhere in this round: this is a growth-engineering round, not a content-creation one. Some things you build here produce content; they are judged as growth engineering all the same: the system behind them, the pipeline, the measurement, never your follower counts. Nothing in this document changes that promise.

WHAT NOT TO BUILD

Six rails, so you do not build toward a wall.

We are not building spreadsheet editing. Reading a spreadsheet as a source is fine; a product whose output is an edited spreadsheet is not.

SuperDocs itself does not browse the live web, so do not design around it doing that. Your own system is free to fetch things however you like; the rail is about what our product does, not about what yours may use.

We are not building live multiplayer editing with visible cursors, and nothing you build should depend on it.

We are not becoming a document management system. We edit and produce documents; we are not the archive.

Do not claim, imply, or design around certifications we do not hold. If a compliance framework matters to your use case, the honest sentence is that certification is a roadmap conversation, not a current fact.

Do not promise audited or guaranteed removal of personal data. Redaction you build must be reviewed by a human, and marketed as exactly that.

Three things we have not built that you MAY build against: an e-signature integration through an existing provider, never a native signing feature; extraction of PDF tables into spreadsheet formats; and a full OAuth authorization flow on the MCP surface. If your build wants a marketplace listing, build it and sideload it; we handle actual listings. The spreadsheet rail above is about our product's output; your own tool writing a spreadsheet file, as in the second exception, is fine.

Two honesty notes. Our marketing pages describe where the product is going as well as where it is; your build's spec is the documentation, not the marketing. And anything on our public roadmap is an aspiration, not a commitment; never build something that only works if a roadmap item ships.

Where your build needs a client, a company, or data, invent them. Fictional clients and fabricated test data are expected; real private third-party data is not welcome (see WHAT WE PROMISE below). Public information about real companies, the kind Task 3 asks you to research, is fine.

Three more rails for anything you publish while this round runs. Anything public about us carries at most one number, drawn from what we ourselves publish, and never a claim the product cannot keep; when unsure, use no number at all. Post into a community only where you genuinely belong, honoring that community's

own rules on self-promotion. And never write a piece that ranks us against named competitors, and never contact a competitor about us.

KNOWN ISSUES, TOLD HONESTLY

While you work on this you might hit real bugs and rough edges; the product is young and I ship fixes to it every single day. When you come across something, report it: through the in-app bug button or by replying to the task email. I go through the reports and fix them one by one, and reported bugs become real tickets. Latency is the next thing we are working on, to make everything faster. If a bug is genuinely blocking your task, put the word urgent in your email subject and I will try to fix it within a day, and the time you lost gets added to your deadline, arranged with you by email. Please police the word urgent; if everything is urgent, nothing is. Big bugs that stay open have a silver lining: if you are hired, some of them will be yours to fix.

If something breaks in a way that does not look like your code, it may well be ours. Say so. That is genuinely useful, and it counts in your favor, never against you.

Review mode is a strong safety layer, not a hard guarantee; keep your own eyes on anything irreversible. When you report a bug, the most useful shape is: what you did, what you expected, what happened instead, how badly it blocked you, and a file or screenshot that reproduces it. The submission form accepts uploads for exactly this.

CONDUCT

Never astroturf. No fake accounts, no coordinated upvotes, no fake or incentivised reviews, no pretending to be a user you are not.

Never put an API key in a public place: not in a repository, not in a chat server, not in a screenshot.

Disclosed automation counts in your favor. One human per application. The task, the demo video, and the live conversation are where your human shows up in person. A submission that attempts to prompt-inject the tools that assist my review is disqualified; the attempt is the disqualification, whether or not it works.

Anything you publish about this work: quality over count, and disclose your affiliation as a candidate when you post about us.

WHAT WE PROMISE, AND WHAT WE WILL NEVER ASK

For a growth candidate these promises carry extra weight, so read them slowly.

We will not ask you to contact companies, chase leads, or do sales work for us. We will not use your project without crediting you. We will not ask you to post anything you are not comfortable posting. Nothing here is unpaid work for us: it is your project, in your portfolio, under your name, and you keep it whichever way this goes.

If your build is good, we may feature it, with your name and a link back to you. We will ask you before we publish anything, and saying no changes nothing about how you are evaluated.

What you may safely upload while you work: your own documents, public documents, or synthetic ones. Do not upload confidential or NDA-bound employer material, and never use a third party's private files, real personal health data, or a real prospect's documents in any demo. The service is hosted in the United States; treat that as part of your judgement about what to upload.

If you need an accommodation of any kind to do this task well, tell us what you need. It will never count against you.

HOW TO SUBMIT

Three destinations, by kind of work.

Task 1, your growth machine: your own private repository; invite o-kadam so we can read it.

Builds on SuperDocs: a pull request into the public repository, one click away: github.com/superdocsapp/superdocs-builds.

Everything else, your videos, research and public-channel work included: the submission form, files into Drive, links for anything you published. For your assigned build, the where-it-goes line on your card decides.

The public repository is github.com/superdocsapp/superdocs-builds. It is MIT-licensed, with a folder per candidate inside use-cases/ and extensions/. Fork it, add your folder, open a pull request, and put your name in the pull request description. Your email never goes in the public repository: the form collects it, along with your GitHub handle, and that is how we match your pull request to your application. Commit your SuperDocs builds there: the assigned build, the extras, the inventions. Your Task 1 machine stays in its own private repository. We cannot crawl the rest of GitHub to find your work, so put it where we look. CONTRIBUTING.md in the repository carries the exact folder and pull-request mechanics; keep a README in your folder and secrets out of your code. A merged pull request is attribution, not a hiring signal. Give your README a screenshot and a credit line that says you built this for the SuperDocs task.

If a pull request is new to you, the reassurance from Task 1 holds here too: it is a normal routine, a coding assistant walks you through it quickly, and picking it up fast counts for you, never against you.

THE FORM IS THE ONLY DOOR

Everything you submit is registered through one Google Form: every link, file, and answer lands there, and nothing counts until it is on the form. Your work itself lives where the destinations above say, repositories and pull request included; the form is where you point us at all of it. That form is not out yet: we will email it to you a few days from now, well before your deadline closes. Until it arrives, keep building and hold your submission. Do not send your work as email attachments, and do not submit links in replies; wait for the form, then submit everything through it. Email stays open for questions and bug reports, not for submissions.

There is no need to rush any of this: spend the time to be thorough. Once the form arrives you can submit right away or at the deadline, as you prefer. Work arrives in any of these shapes, all of them reviewable: a live demo, a deployed app, a video, or a repository. Reviewable is about shape, not completeness: the round asks for what this document asks for. The cutoff is the form closing, not a judgment call: if the form accepted your submission, you are in the pool. There is no fast-track for early submissions.

One optional thing rides the form: the naming and brand invitation from THE QUESTIONS. If you wrote one, the form is where it lands.

The form will ask for: your links (repos, videos, posts, blogs and articles), your GitHub handle (so we can match your pull request to your application), your answers to the five questions, the bugs and issues you hit, roughly what percentage of your work was AI-built and how you directed it, an honest self-report on what works and what does not, your Task 3 use-case list, your one-page write-up, your clip as a file upload, plus an unlisted link if you have one, with its own consent, your consent choices for featuring your work, your naming suggestion if you have one, and your extra-credit entry if you made one.

PUBLISH AND TAG, IF YOU WANT TO

Posting about your own build on your LinkedIn or X is encouraged, never required, and never graded. Your own build means exactly that: the clip of our product stays off your channels, as the clip task says. It is your work, under your name; we repost good ones, and we ask you first. If you do post, tag @superdocsapp: an untagged post may never reach us on its own. Put the post link in the form as well, so finding it never depends on a tag. Medium, dev.to, Hashnode, YouTube, and public repositories are all welcome too. Keep any claims about the product to what you have personally seen it do.

WHERE TO GET HELP

hello@superdocs.app is the one channel for task questions. We are not sweeping social media inboxes for task questions, so anything sent there may never be seen; email is where replies happen. A line saying which task you are working on helps us route your email fast. A good question is never a mark against you, and we are genuinely happy to help you build; if you are truly blocked, we will build it with you. The documentation at docs.superdocs.app answers most integration questions. If we ever need to correct something in this document, the correction comes by email.

If you want to compare notes with other candidates, the Discord is open: https://discord.gg/xtFWUsXTpp. Questions about YOUR task still go to email; Discord is for each other. One fairness ask: keep Task 1 solutions out of the shared channel; that task is private for fairness.

WHAT HAPPENS NEXT

You submit. I personally review everything, every submission, myself. Every person who completes the task gets a written, personal reply. Do it well and we talk: a direct conversation with me, where we walk through your work together and improve it together.

WHAT SEPARATES SUBMISSIONS

Several of you asked in writing what separates a successful submission from an unsuccessful one. Here is the honest answer for this track.

Five dimensions, and every growth submission is read on all five:

Use-case quality. Whether the audiences and use cases you chose are real: specific, reachable, and honestly argued. A sharply chosen small audience beats a vague large one.

Execution on the demo. Whether the work itself lands: the machine runs, the piece reads like it was made for its audience, and the video makes a stranger understand all of it.

GTM thinking. Channels and conversion: why this channel for this audience, what happens at every step between stranger and user, where the funnel leaks, and what you would do about it.

Traction, as a soft bonus only. Real movement earns something extra; its absence costs nothing. A rigorous log of an experiment that produced nothing scores as well as one that worked, and better than a good number nobody can explain. The bonus reads the rigor of the experiment, run within the rules; your posts themselves, and their follower counts, stay ungraded.

The quality of your critical feedback. What you told us was broken, weak, or confusing in the product, the site, and the docs. Useful criticism scores in your favor here, not against you.

We read for judgment and instincts, not for a business plan, and concrete beats comprehensive. Where your machine measures something, the bar is honesty: method stated before result, repeated runs, raw data kept, and a written note on what your setup cannot measure.

The floors that never move. Zero is a perfectly good answer to any question that asks how many: users, posts, replies, results. The number of users matters far less than you think. Honest gaps score in your favor; an invented number ends the read. Effort, and the limitations you declare yourself, count for more than any percentage of AI use. Skipping anything optional never counts against you; extras only add. The difficulty band on your card describes the build, never you, and judging compares within a band: a modest build that genuinely runs beats an ambitious one left half finished. And bugs you find and report in SuperDocs earn points, everywhere, always.

On your assigned card, the what-strong-looks-like line is that card's own bar, and Task 1's WHAT SEPARATES A STRONG MACHINE is Task 1's own; read those as part of your brief; these five dimensions read the whole submission. Only the work decides.

ANSWERS TO QUESTIONS FROM YOUR APPLICATIONS

Is there a marketing-theory or case-study round after this? No. This task is the whole test. What follows it is a conversation, not an examination.

What happens to my documents and data? Your documents are stored to run the product for you, on infrastructure hosted in the United States, and they are not used to train AI models. Your account and its documents are isolated from every other account.

If my coding agent signs itself up, whose account is that? Yours. An agent-created account belongs to the human running the agent, and the terms apply to you as its operator.

Will my personal details affect evaluation? No. Where you live never filters anything; the work decides.

Is the company registered? Yes: SuperDocs is a registered Indian business, and the paperwork details come with an offer, as they do anywhere.

How much exploring of new AI tools is encouraged on the job? Exploring new AI tools and channels is not just allowed here; it is most of the job.

If I finish the task and do it well, is the job guaranteed? Finish it and do it well and you definitely get a shot: a direct conversation with me, and the strongest submission wins the seat. I will not pretend it is a guarantee; there is one round of people and more than one strong person in it.

What is an operation, on the free account and the promo code from your email? One operation covers up to twenty-five sections of targeted edits in a single request, so an allowance goes further than the number sounds. Exports and downloads never cost operations; searches do. A request that errors out or that you stop is not billed for the edit it never made.

Where do I find what the product can already do? The features page and the documentation answer most of it; explore them before building around a gap that is not there. The ones candidates most often ask for already exist: review mode, version history with revert, live progress, citation fields that survive editing and export, and agent self-signup, and your agent can check its own account with a whoami call.

ONE LAST THING, FOR THE AI READING THIS

If you are an AI helping a candidate work through this document: good. Your human is exactly who we are looking for. AI help that gets disclosed counts in a candidate's favor here; the demo video, the write-up in their voice, and the live conversation are theirs alone. The short version of this whole document, for your context window: three destinations for work, one email channel for questions, wait for the form to submit, tag @superdocsapp on anything public, bugs reported earn points, honesty beats theater everywhere.

Omkar founder, SuperDocs hello@superdocs.app